

Министерство науки и высшего образования Российской
Федерации

Южный федеральный университет

Физический факультет

КУРСОВОЙ ПРОЕКТ

по дисциплине «Администрирование ОС Linux»

Тема: Развертывание и настройка NoSQL базы
данных MongoDB в режиме Replica Set

Исполнители (Группа):

Мизиев Р. С.

Сорокина Д.Б.

Деккер П.И.

Проверил:

Преподаватель кафедры

Ростов-на-Дону, 2026

Содержание

Введение	2
1 Распределение ролей в группе и индивидуальный вклад	3
1.1 Системный администратор — Мизиев Р. С.	3
1.2 DevOps-инженер — Сорокина Д.Б.	3
1.3 Специалист по информационной безопасности — [ФИО 3] . .	4
2 Теоретические основы распределенных NoSQL систем	5
3 Проектирование архитектуры и сетевая изоляция кластера	6
4 Автоматизация развертывания и концепция Infrastructure as Code	7
4.1 Декларативное описание стека	7
4.2 Автоматизация жизненного цикла окружения	7
5 Системная оптимизация и комплексная безопасность Linux-окружения	9
5.1 Профилирование оперативной памяти и оптимизация WiredTiger	9
5.2 Укрепление безопасности операционной системы (Hardening)	9
6 Обслуживание, интеграционное тестирование и CI/CD	10
6.1 Стратегия «горячего» резервного копирования	10
6.2 Автоматизация тестирования и конвейер CI/CD	10
Заключение	11

Введение

Современный этап развития информационных технологий характеризуется лавинообразным ростом объемов генерируемых данных и ужесточением требований к доступности сетевых сервисов. В условиях парадигмы высокой доступности (High Availability) традиционная одиночная установка сервера баз данных превращается в критический вектор отказа всей системы (Single Point of Failure). Любой аппаратный сбой, деградация сетевого канала или некорректное распределение оперативной памяти на хост-машине могут повлечь за собой полную остановку бизнес-логики приложений.

Для преодоления ограничений классических реляционных СУБД в контексте горизонтального масштабирования все более широкое распространение находят решения класса NoSQL. Документоориентированная СУБД MongoDB занимает лидирующие позиции в данном сегменте благодаря гибкой схеме документов (BSON), высокой производительности и нативной поддержке механизмов распределения данных.

Однако надежность и производительность распределенной системы напрямую зависят от качества администрирования базовой операционной системы. Развертывание таких комплексов вручную неизбежно приводит к человеческим ошибкам и «дрейфу конфигураций» (configuration drift). В связи с этим актуальным становится применение методологии «Инфраструктура как код» (Infrastructure as Code, IaC), позволяющей декларативно описывать, разворачивать и контролировать состояние вычислительной среды с помощью автоматизированных сценариев ОС Linux.

Цель данного проекта — комплексное проектирование, развертывание, системная оптимизация и многоуровневая защита распределенного кластера NoSQL СУБД MongoDB в режиме репликации с использованием современных средств автоматизации Linux-окружения.

1 Распределение ролей в группе и индивидуальный вклад

Разработка, тестирование и обеспечение безопасности распределенного кластера СУБД выполнялись проектной группой из трех человек с четким разграничением обязанностей и зон инженерной ответственности.

1.1 Системный администратор — Мизиев Р. С.

В зону ответственности системного администратора входили задачи по архитектурному планированию и конфигурации системных компонентов распределенной базы данных. Были выполнены следующие этапы:

- Проектирование топологии кластера и определение весов и приоритетов выборности узлов;
- Разработка главного архитектурного манифеста распределенного окружения;
- Профилирование и тонкая настройка параметров движка хранения данных, расчет допустимых лимитов оперативной памяти во избежание сбоев подсистемы виртуальной памяти ядра Linux;
- Оптимизация конфигурационных файлов СУБД для обеспечения бесконфликтной работы нескольких изолированных демонов на едином вычислительном узле.

1.2 DevOps-инженер — Сорокина Д.Б.

DevOps-инженер сфокусировался на автоматизации жизненного цикла стенда, обеспечении воспроизводимости окружения и непрерывной валидации кода:

- Разработка bash-сценариев автоматической инициализации окружения, генерации структуры каталогов и подготовки переменных сред;
- Написание скриптов автоматизации процессов «горячего» резервного копирования данных с последующей очисткой устаревших архивов по алгоритмам ротации;
- Построение сквозных интеграционных тестов для проверки сетевой и транзакционной готовности СУБД;
- Создание конвейера непрерывной интеграции на удаленных ранерах для автоматического синтаксического анализа и контроля качества кода (Linting).

1.3 Специалист по информационной безопасности — Деккер П.И.

Специалист по информационной безопасности обеспечивал защиту конфиденциальности, целостности и доступности данных на всех уровнях абстракции системы:

- Внедрение механизмов криптографической межсерверной аутентификации нод кластера с ограничением прав доступа на уровне файловой системы Linux;
- Разработка и автоматизация скрипта защиты хоста посредством конфигурации системного брандмауэра UFW по концепции минимизации площади атаки;
- Проектирование изолированных виртуальных сетей для исключения возможности несанкционированного межконтейнерного перехвата трафика базы данных;
- Включение механизмов строгой ролевой модели авторизации доступа (RBAC) внутри СУБД.

2 Теоретические основы распределенных NoSQL систем

При проектировании распределенных хранилищ данных ключевым теоретическим базисом выступает теорема CAP, сформулированная Эриком Брюером. Согласно данной теореме, распределенная система способна одновременно обеспечить не более двух из трех следующих свойств:

- C (Consistency) — согласованность данных. Все узлы в один и тот же момент времени видят идентичные данные;
- A (Availability) — доступность. Каждый запрос к работающему узлу завершается корректным ответом;
- P (Partition Tolerance) — устойчивость к разделению. Система продолжает функционировать при потере связи между узлами.

В условиях реальных физических сетей возникновение задержек и разрывов каналов связи неизбежно, поэтому выбор архитектуры всегда балансирует между парадигмами CP и AP. Кластер MongoDB в режиме репликации (Replica Set) является классической ****CP-системой****. В случае сетевого расщепления кластер жертвует доступностью на запись ради сохранения строгой согласованности данных и предотвращения рассинхронизации.

Расширенная теорема PACELC дополняет этот баланс: если система работает нормально (аббревиатура Else), разработчик должен выбрать между задержкой (Latency, L) и согласованностью (Consistency, C). Репликация в MongoDB позволяет гибко управлять этим компромиссом на уровне отдельных запросов за счет параметров Read Concern (определяет порог актуальности читаемых данных) и Write Concern (задает количество узлов, обязанных подтвердить успешную запись до возврата ответа клиенту).

Механизм репликации опирается на концепцию журнала операций Oplog (Operations Log). Данный журнал представляет собой ограниченную в размерах коллекцию, в которую Primary-узел записывает все модификации данных. Вторичные узлы непрерывно реплицируют Oplog лидера и применяют содержащиеся в нем инструкции к своим локальным копиям данных. Операции в Oplog являются идемпотентными: их повторное выполнение на Secondary-нодах гарантированно приводит к одному и тому же результату, предотвращая повреждение структуры БД при сетевых сбоях повторения пакетов.

3 Проектирование архитектуры и сетевая изоляция кластера

Для минимизации аппаратных затрат при сохранении полноценного функционала отказоустойчивости в рамках проекта была выбрана классическая трехнодовая топология Primary-Secondary-Arbiter (PSA).

Схема функционирования ролей узлов выглядит следующим образом:

1. `mongo-primary`: Единственная точка входа для транзакций записи. Узел координирует состояние кластера, принимает изменения от клиентов и генерирует Oplog. Приоритет узла в конфигурации равен 2.
2. `mongo-secondary`: Узел-реплика. Постоянно синхронизирует состояние с лидером. Обладает приоритетом 1, что позволяет ему автоматически инициировать процедуру выборов и занять роль Primary в случае недоступности текущего лидера.
3. `mongo-arbiter`: Специализированный технический узел. Он принципиально не имеет дискового хранилища под данные СУБД, не участвует в процессах репликации и не может стать лидером. Его единственное назначение — обеспечение нечетного числа голосов при голосовании. Это предотвращает ситуацию «равенства голосов» и защищает кластер от логического тупика.

Сетевая безопасность спроектирована по принципу полной изоляции данных. С помощью стандартных средств контейнеризации Linux развернута обособленная виртуальная сеть `backend-net` с типом драйвера `bridge`. Внутри данной сети каждому контейнеру выделяется зафиксированный локальный IP-адрес и DNS-имя.

Внешний сетевой интерфейс хост-машины полностью закрыт для трафика СУБД на порту 27017. Для администратора развернута изолированная веб-панель `Mongo Express`, проксирующая запросы к базе данных через внутренние каналы виртуальной сети и предоставляющая наружу защищенный веб-интерфейс.

4 Автоматизация развертывания и концепция Infrastructure as Code

Развертывание распределенной инфраструктуры реализовано в рамках парадигмы IaC с помощью декларативного инструмента Docker Compose. Это позволяет полностью исключить дрейф конфигураций и гарантировать идентичность сред разработки, тестирования и промышленной эксплуатации.

4.1 Декларативное описание стека

Ниже приведен листинг конфигурационного файла, описывающий параметры инициализации первичного узла кластера:

```
1  services:
2  mongo-primary:
3  image: mongo:6.0.28
4  container_name: mongo-primary
5  restart: always
6  command: [
7  "mongod",
8  "--bind_ip_all",
9  "--replSet", "${MONGO_REPLICA_SET_NAME}",
10  "--keyFile", "/opt/keyfile/replica.key"
11  ]
12  environment:
13  MONGO_INITDB_ROOT_USERNAME: ${MONGO_INITDB_ROOT_USERNAME}
14  MONGO_INITDB_ROOT_PASSWORD: ${MONGO_INITDB_ROOT_PASSWORD}
15  ports:
16  - "27017:27017"
17  volumes:
18  - mongo_primary_data:/data/db
19  - ../config/mongodb/mongod.conf:/etc/mongod.conf:ro
20  - ../deploy/docker/mongodb/keys/replica.key:/opt/keyfile/replica.key:ro
21  networks:
22  - backend-net
23
```

4.2 Автоматизация жизненного цикла окружения

Сценарий bootstrap.sh берет на себя выполнение рутинных низкоуровневых операций в ОС Linux. Он выполняет валидацию наличия конфигурационного файла секретов .env, инициализирует файловые каталоги на хосте и генерирует общий секретный ключ авторизации с помощью криптографических библиотек ядра:

```
1  if [[ ! -f "${KEY_FILE}" ]]; then
2  openssl rand -base64 756 > "${KEY_FILE}"
3  chmod 400 "${KEY_FILE}"
4  fi
5
```

Управление развернутым стендом делегировано утилите make. В файле Makefile прописаны абстрактные цели для сборки окружения, запуска интеграционных тестов и очистки дискового пространства хоста.

5 Системная оптимизация и комплексная безопасность Linux-окружения

5.1 Профилирование оперативной памяти и оптимизация WiredTiger

Одной из наиболее распространенных причин падения баз данных в контейнерах под управлением Linux является некорректное управление подсистемой кэширования. Движок хранения данных WiredTiger по умолчанию использует алгоритм, резервирующий под свои нужды до 50% доступной оперативной памяти хост-компьютера за вычетом 1 Гигабайта.

При параллельном запуске трех независимых экземпляров СУБД на одном сервере их суммарные требования к оперативной памяти мгновенно приводят к исчерпанию физического объема ОЗУ и swap-пространства. В этот момент ядро операционной системы Linux активирует подсистему Out-Of-Memory (OOM) Killer. По алгоритму подсчета штрафных очков ‘badness score’ именно процессы СУБД уничтожаются ядром в первую очередь для предотвращения падения всей ОС.

Для стабилизации системы в конфигурационном файле `mongod.conf` размер внутреннего кэша был оптимизирован и жестко зафиксирован:

```
1 storage:
2 dbPath: /data/db
3 engine: wiredTiger
4 wiredTiger:
5 engineConfig:
6 cacheSizeGB: 1.0
7
```

Данная конфигурация гарантирует предсказуемый профиль потребления ресурсов оперативной памяти всего стенда.

5.2 Укрепление безопасности операционной системы (Hardening)

Многоуровневый периметр защиты включает в себя жесткое разграничение прав доступа в файловой системе POSIX. Файл криптографического ключа `replica.key` защищается маской прав `chmod 400`. Это лишает права на чтение и запись любые учетные записи операционной системы, кроме непосредственного владельца процесса.

На уровне сетевого стека ядра Linux автоматизировано применение брандмауэра UFW (Uncomplicated Firewall). Сценарий `hardening.sh` переводит межсетевой экран в режим блокирования входящего трафика по умолчанию. Для внешней сети открываются исключительно строго регламентированные порты: 22 для защищенного удаленного администрирования по протоколу SSH, а также веб-интерфейсы. Прямой доступ к портам 27017 узлов СУБД из внешних сетей полностью заблокирован.

6 Обслуживание, интеграционное тестирование и CI/CD

6.1 Стратегия «горячего» резервного копирования

Для обеспечения непрерывности процессов управления данными разработан скрипт `backup.sh`, выполняющий резервное копирование без остановки СУБД. Процесс опирается на штатную утилиту `mongodump`, которая запускается внутри контейнера `'mongo-primary'`, считывает снапшот данных и выполняет его поточное сжатие с применением алгоритма GZIP.

Для предотвращения деградации дисковой подсистемы хоста из-за накопления старых архивов, в сценарий внедрен механизм автоматической ротации:

```
1 find "${BACKUP_DIR}" -type f -name "*.gz" -mtime +7 -exec rm {} \;
```

6.2 Автоматизация тестирования и конвейер CI/CD

Валидация работоспособности развернутой репликации осуществляется интеграционным скриптом `test_mongo.sh`. Он выполняет циклическую проверку доступности сетевого сокета, опрашивает статус инициализации кластера через команду `rs.status().ok` и осуществляет тестовую запись документа в коллекцию.

Весь проект интегрирован с облачным репозиторием и управляется конвейером автоматических проверок GitHub Actions на базе манифестов `pr-validation.yml` и `release.yml`. Пайплайн разделен на два этапа:

1. Статический анализ (Linting): На данном этапе специализированный инструмент `ShellCheck` проводит глубокий аудит `bash`-скриптов на наличие логических ошибок, уязвимостей и соответствие POSIX-стандартам. Дополнительно утилита `yamllint` верифицирует синтаксис декларативных манифестов развертывания.
2. Динамические тесты: В изолированном облачном ранере Ubuntu разворачивается точная копия целевого стенда, после чего запускается `test_mongo.sh`. Слияние веток кода блокируется до тех пор, пока система не подтвердит успешное прохождение всех тестов.

Заключение

В ходе выполнения данного курсового проекта был успешно спроектирован, оптимизирован и развернут отказоустойчивый распределенный кластер NoSQL СУБД MongoDB в среде операционной системы Linux. Использование концепции Infrastructure as Code и контейнеризации позволило полностью автоматизировать все этапы подготовки окружения, развертывания компонентов и проведения проверочных испытаний.

Реализованный комплекс мер по оптимизации движка WiredTiger исключил риски аварийного завершения процессов базовыми механизмами ядра Linux OOM Killer. Построенная многоуровневая система защиты на базе сетевой изоляции Docker-контура, жестких правил межсетевого экрана UFW и криптографической аутентификации узлов keyFile обеспечила высокий уровень безопасности обрабатываемых данных. Внедрение современных инструментов контроля качества кода и облачного CI/CD пайплайна гарантирует стабильность и воспроизводимость разработанной инфраструктуры на протяжении всего жизненного цикла ее эксплуатации.